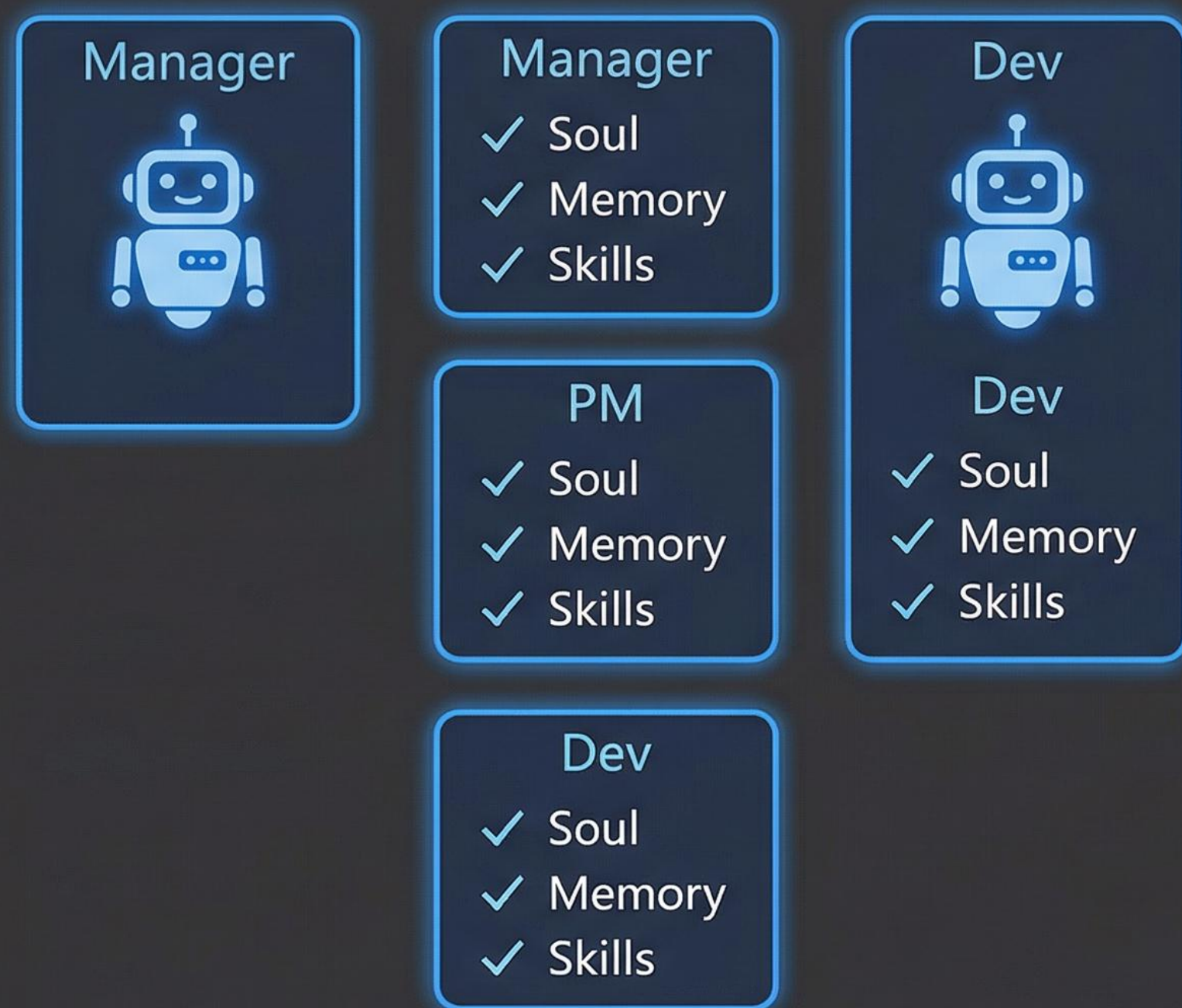


## 26 | 任务链与信息传递

### 数字员工的协作协议

# 角色有了，但协作没有

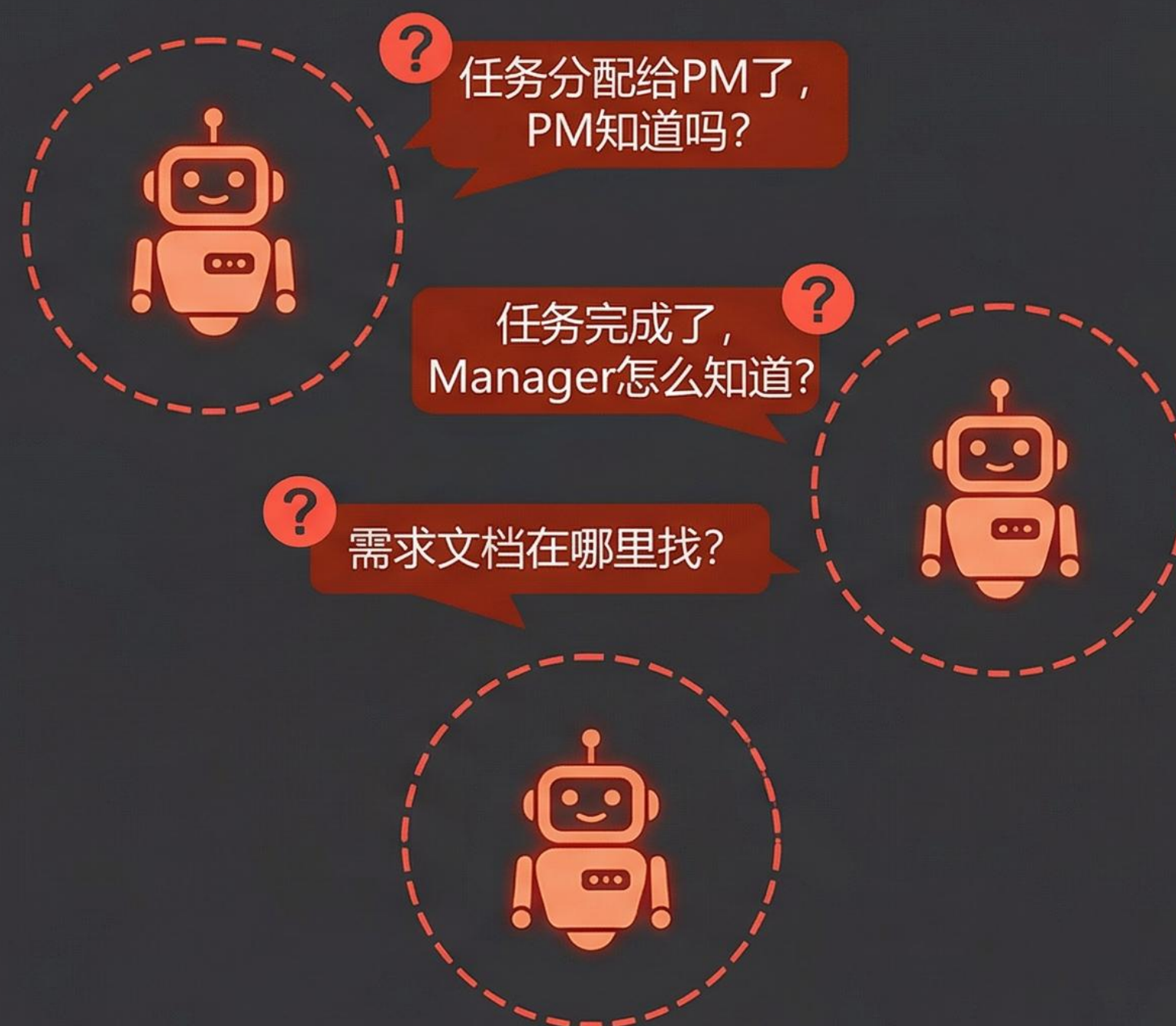
## 第25课完成的工作



每个人都准备好了

本课解决两个独立问题：① 状态共享——谁能看到什么 ② 任务触发——谁去通知谁

## 仍然缺失的问题

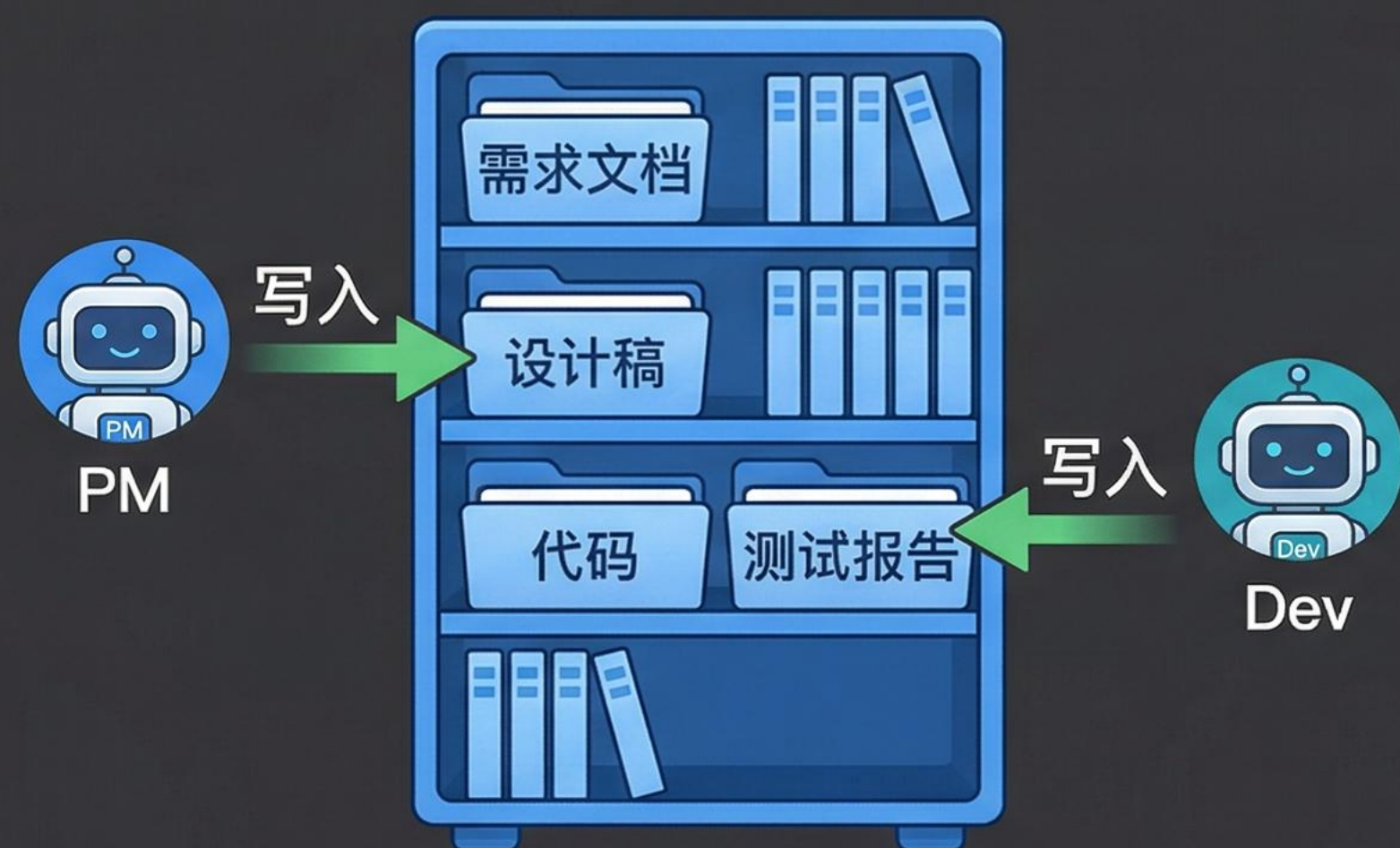


但他们互相不认识，也不知道彼此在做什么



# 人类项目协作的两种机制

## 机制一：公共区域（状态共享）



结构化存储 / 按权限访问  
/ 不需要问人就能找到文档

## 机制二：消息/邮件（任务殷发）

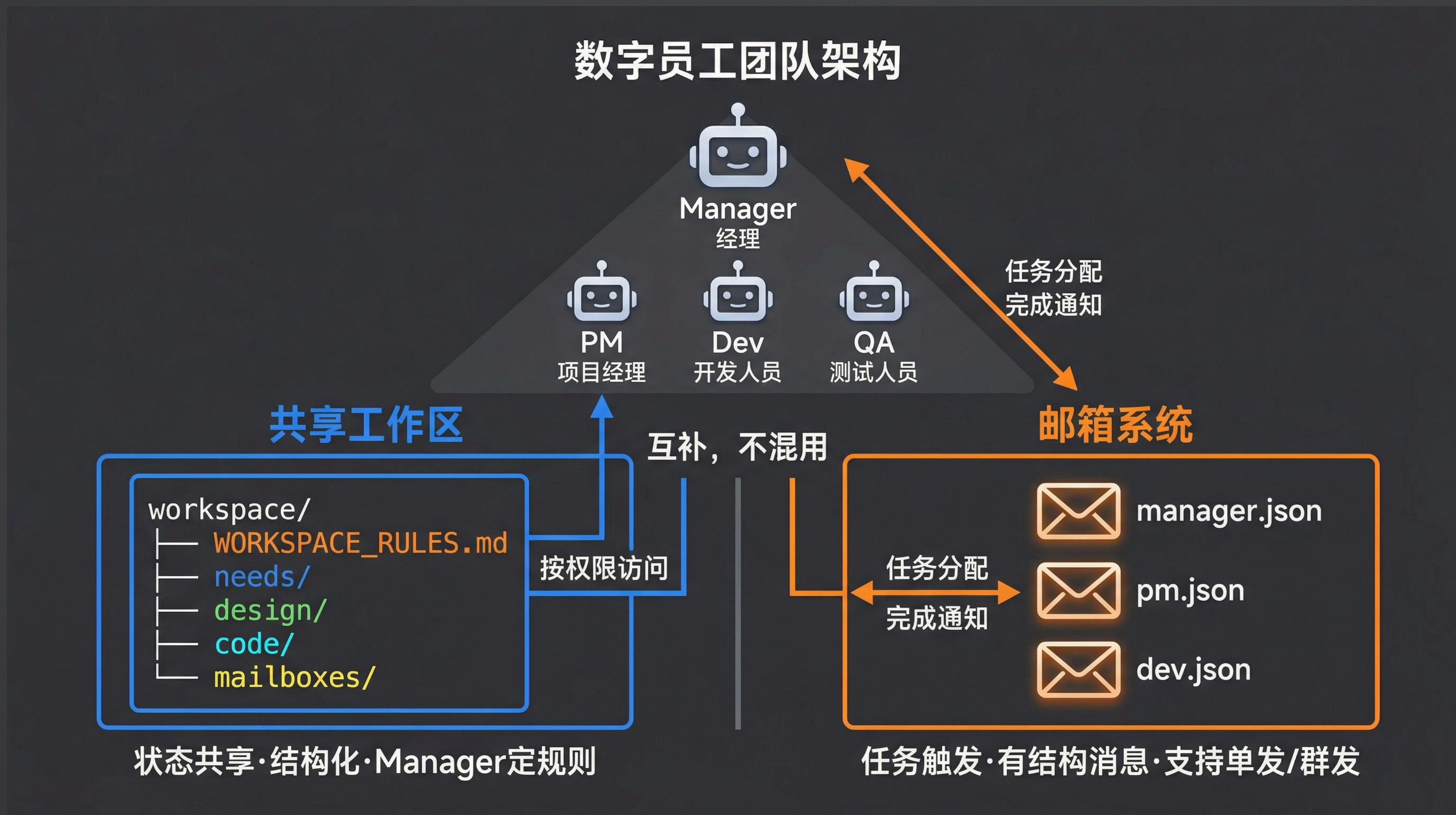


有明确结构 / 支持单发/群发  
/ 任务完成后回复通知

两种机制解决不同问题，不要混用——工作区不能发通知，邮件不能存文档



# 共享工作区解决"状态在哪里", 邮箱解决"谁让我做什么"





# 共享工作区：结构化、有权限、Manager定规则

## 工作区结构

```
workspace/
├── WORKSPACE_RULES.md
├── needs/
│   └── requirements.md
├── design/
│   └── product_spec.md
├── code/
└── qa/
```

## 角色权限表

...

| 角色      | needs/ | design/ | code/ | qa/ |
|---------|--------|---------|-------|-----|
| Manager | ✓读写    | ✓读写     | ✓读写   | ✓读写 |
| PM      | ✓读     | ✓写      | ✗禁止   | ✗禁止 |
| Dev     | ✓读     | ✓读      | ✓写    | ✗禁止 |
| QA      | ✓读     | ✓读      | ✓读    | ✓写  |

规范来源

WORKSPACE\_RULES.md

规范来源

Manager制定规范 → 打包成 reference skill

PM拿到的 reference skill 只告诉它：读 needs/，写 design/——注意力自动聚焦

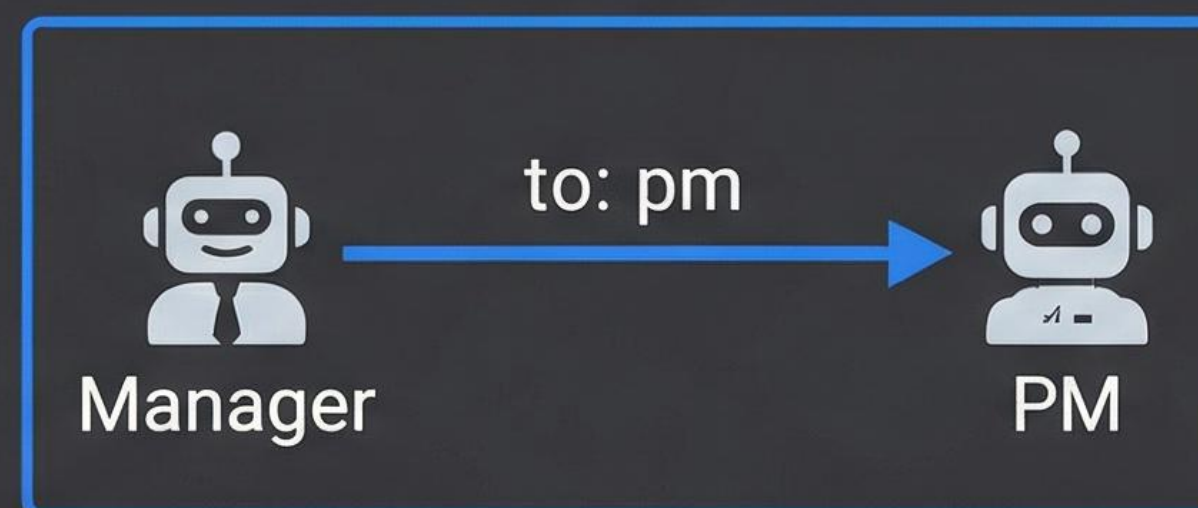


# 邮箱设计：有结构的消息系统

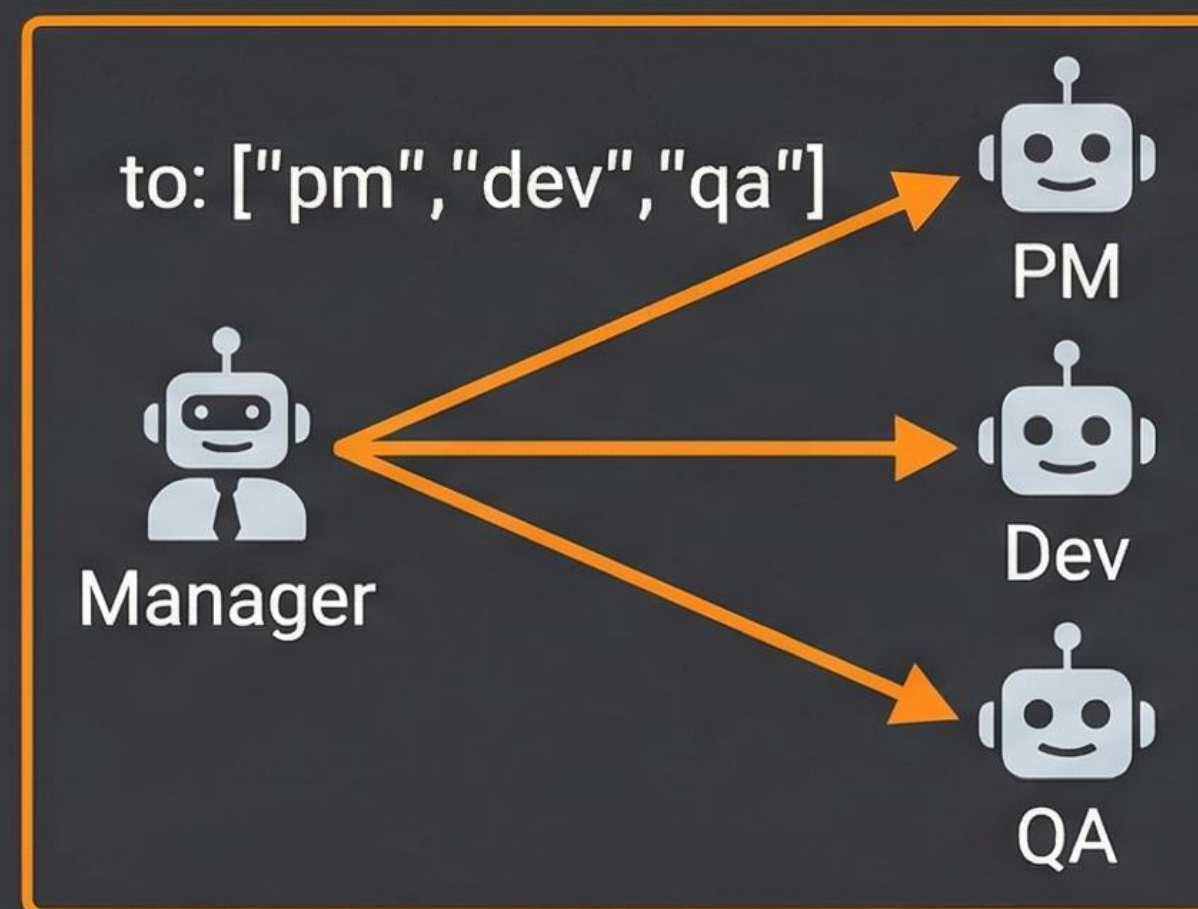
```
{  
  "id": "msg_001",  
  "from": "manager",  
  "to": "pm",  
  "type": "task_assign",  
  "subject": "产品文档设计",  
  "content": "请根据需求文档...",  
  "timestamp": "2026-04-10T...",  
  "read": false  
}
```

type → 接收方知道如何处理，无需 LLM 推理  
read → 幂等保护，防止同一消息被处理两次

## 单发



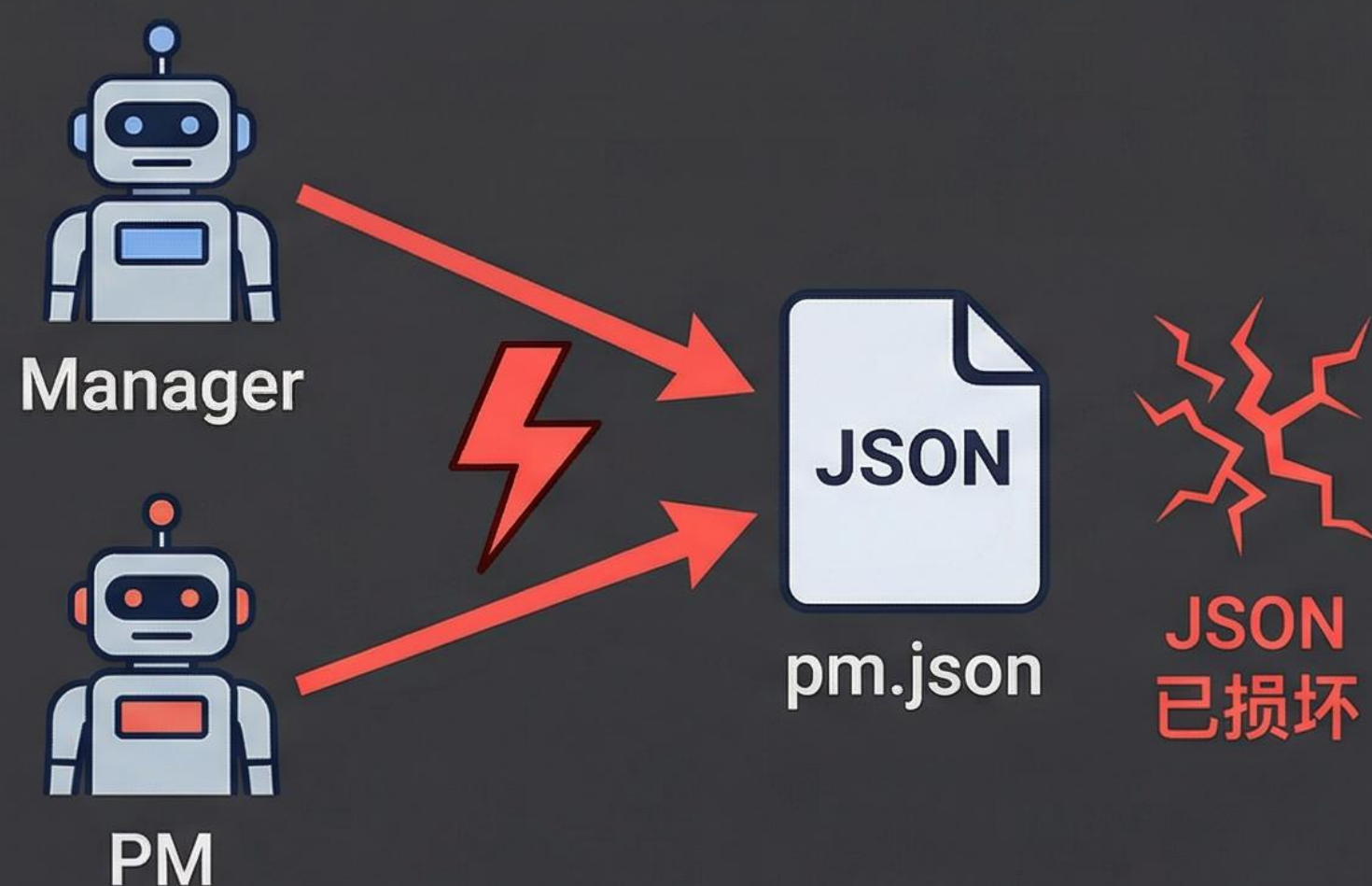
## 群发





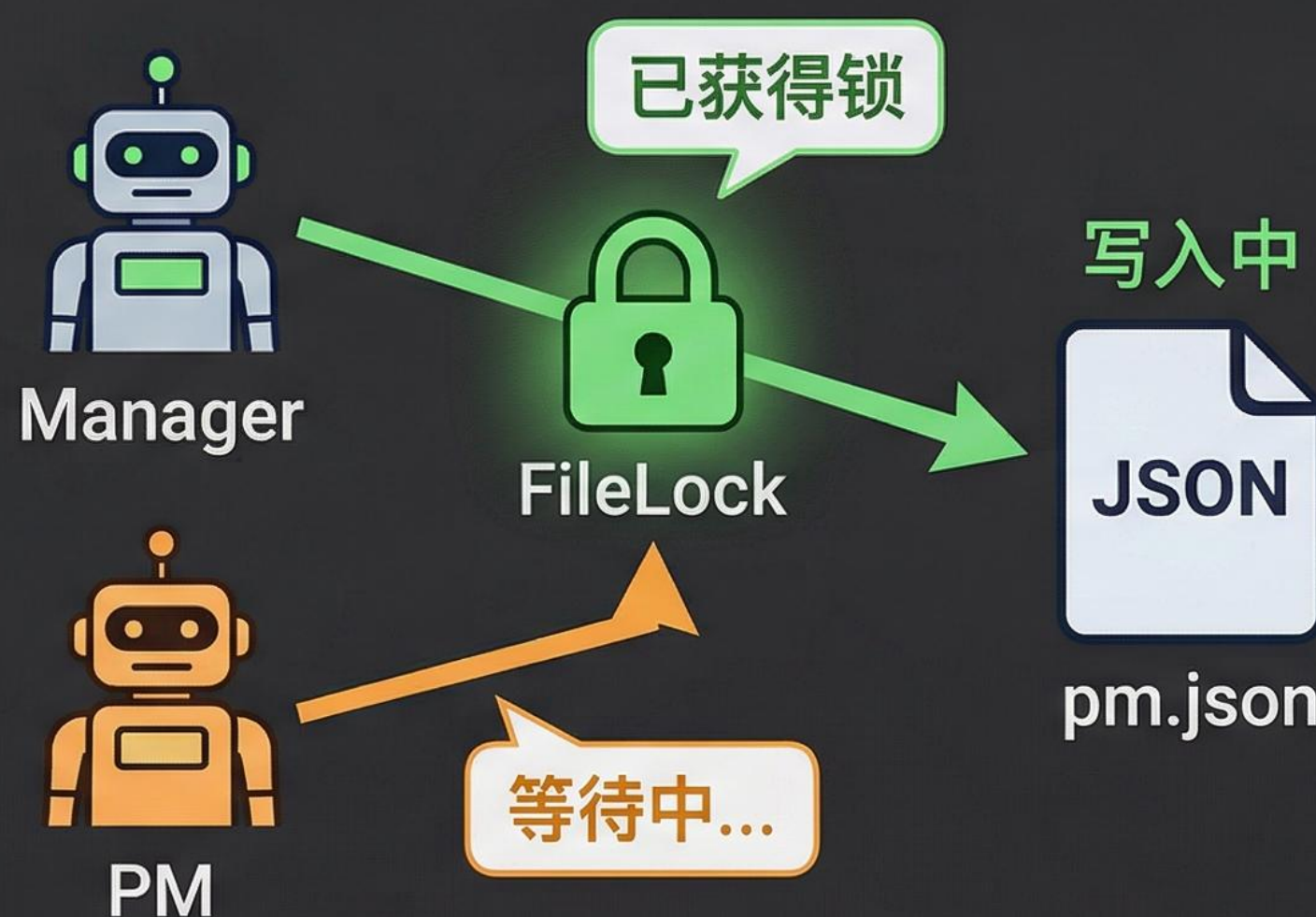
# 代码实战：邮箱实现：公共 Skill + 文件锁

## ✗ 无文件锁



两个 Agent 同时写入，文件格式损坏

## ✓ FileLock 保护



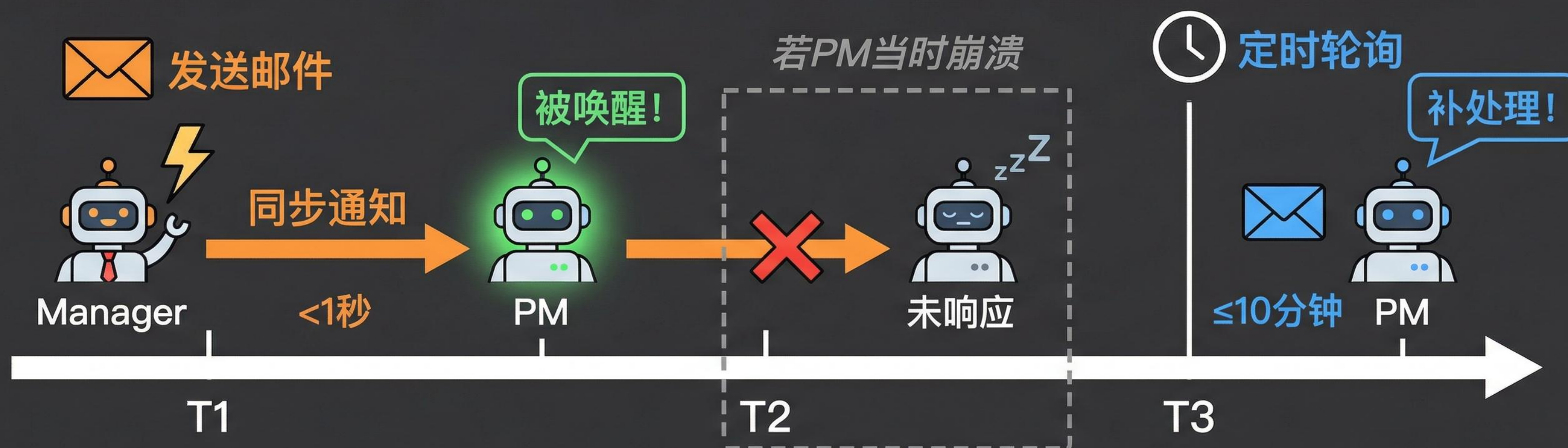
```
with FileLock("pm.json.lock"):
    messages = read()
    messages.append(new_msg)
    write(messages)
```

读+写在同一锁内

mailbox-ops Skill 封装了这段逻辑——Agent 只需调用 send\_mail，不关心底层锁



## 同步通知 + 后台唤醒：两种机制互斥互补



### 同步通知（主动唤醒）

- ✓ 及时，发邮件即触发
- ✓ 延迟极低
- ✗ PM崩溃则消息可能丢失

互补

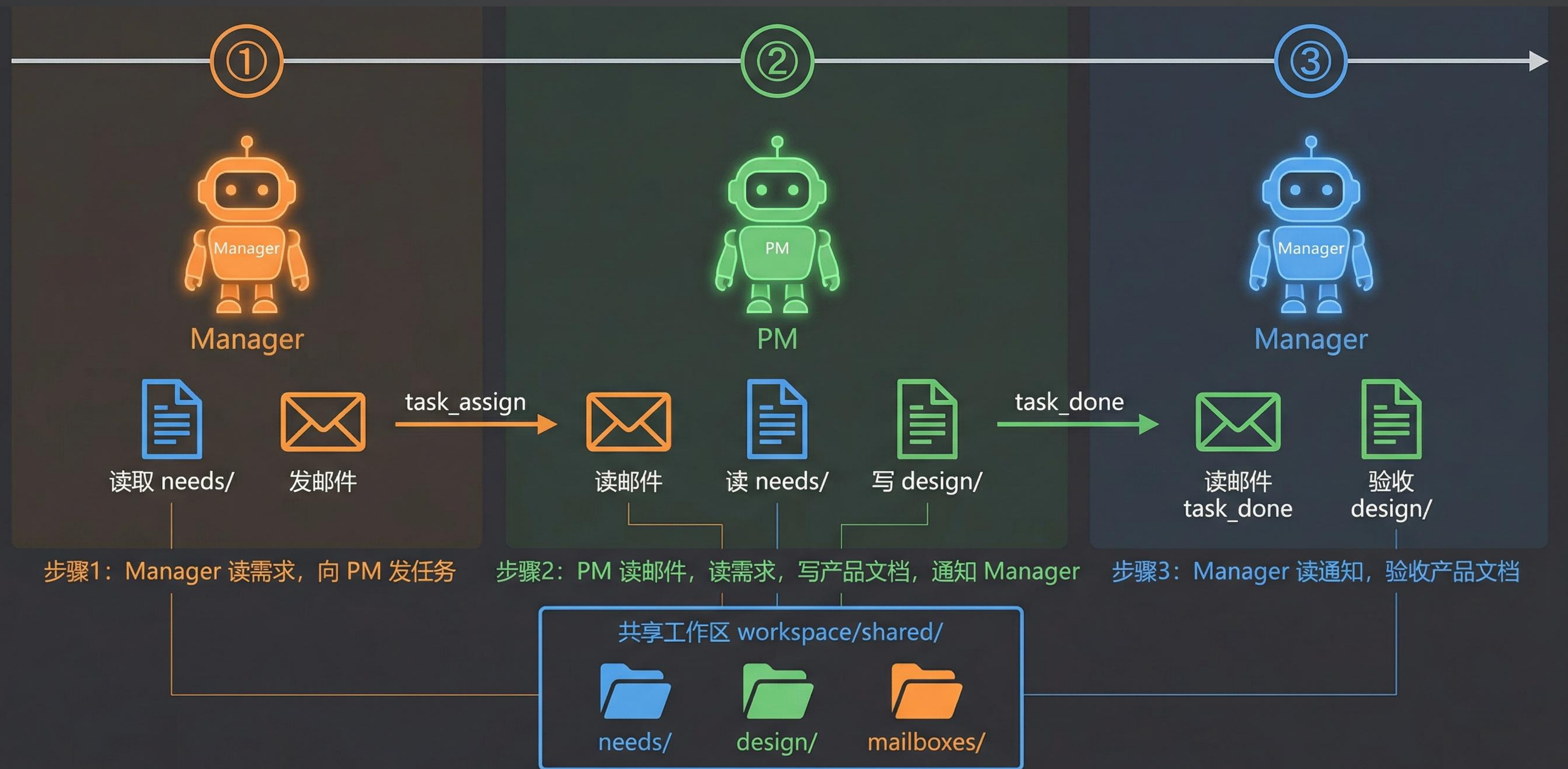
### 后台唤醒（定期轮询）

- ✓ 兜底，确保不遗漏
- ✓ read:true 防重复处理
- ✗ 最晚等一个轮询周期

两个机制一起用：平时靠同步通知，定时任务兜底——消息既快又不丢



# Manager → PM → Manager 三步任务链



顺序固定 (Manager→PM→Manager) + 内容动态 (AI生成) + 步骤间结构检查



# 最佳实践与反模式

## 反模式

---

多 Agent 并发写同一文件，没有文件锁

消息丢失是静默的——LLM 不会报错，Agent

不知道消息没到，整个任务链在某一步卡死

工作区没有 owner

MAST 分类法研究表明，所有 Agent 失败模式

37%的根因是资源所有权不唯一

## 最佳实践

---

文件的 rename 安全写入

防止并发协作时冲突，写文件后 rename

Agent协作通过文件 + 轻量级触达

大幅度节省 token 成本

对邮件消息处理的三态状态机

初始->已读->已处理，可靠性设计保证每个消息都正

确处理，不会一直 hang 住



# 课程总结

共享工作区和邮箱两种 agent 协作方式

共享工作区需要 manager 创建并定义好权限分工

邮箱需要结构化，通过同步通知和主动唤醒来完成协作

下节提示：人和数字团队应该如何协作